

Optimization for Machine Learning: Placement Guide

Optimization chooses parameters that make a model's predictions useful. Let $D=\{(x_i, y_i)\}_{i=1}^n$, parameters be θ , and empirical objective be $J(\theta)=(1/n) \sum_i \ell(f_\theta(x_i), y_i)$. In practice, optimize a training objective, validate hyperparameters on held-out data, and report the metric the product actually cares about.

Loss Functions

1. Overview

A loss function converts a prediction and target into a non-negative training signal. Training minimizes average loss; evaluation may use a different business metric. Losses drive regression, classification, detection, language modelling, and generative models.

2. Intuition

Loss is a penalty score: predicting a house price far from its label should cost more; assigning a spam email a tiny spam probability should cost heavily. The optimizer follows the direction that reduces the score.

3. Prerequisites

Functions, derivatives, probability, expectation, logits, sigmoid/softmax, and train/validation/test splits.

4. Core Concepts

- **Per-example versus batch loss:** ℓ_i is one example's penalty; $J=(1/n)\sum \ell_i$ is optimized. Averaging makes gradients comparable across batch sizes. Interview: distinguish loss from accuracy.
- **Regression losses:** MSE is sensitive to outliers; MAE is robust but has a non-smooth point at zero. Choose according to error cost and noise.
- **Classification losses:** cross-entropy compares predicted probabilities with labels and strongly penalizes confident wrong predictions. Feed logits, not already-softmaxed values, to numerically stable library APIs.
- **Regularization:** $J_{\text{total}}=J_{\text{data}}+\lambda R(\theta)$ trades fit for simpler parameters. It is not a substitute for validation or correct preprocessing.

5. Algorithm / Working Process

For each batch: run the model to get predictions/logits, compute loss against labels, backpropagate $dJ/d\theta$, update parameters, then monitor validation loss and task metrics. At inference, loss is normally not needed; convert scores to the required output.

6. Mathematical Foundation

Regression: $MSE = (1/n) \sum (y_{\hat{i}} - y_i)^2$, $MAE = (1/n) \sum |y_{\hat{i}} - y_i|$, and Huber loss is $0.5r^2$ for $|r| \leq \delta$, otherwise $\delta(|r| - 0.5\delta)$, where $r = y_{\hat{i}} - y_i$.

Binary cross-entropy: $BCE = -(1/n) \sum [y \log p + (1-y) \log(1-p)]$, $p = \text{sigmoid}(z)$. Multiclass cross-entropy: $CE = -\sum_k y_k \log(\text{softmax}(z)_k)$, where $\text{softmax}(z)_k = e^{z_k} / \sum_j e^{z_j}$. For softmax-CE, $dL/dz_k = p_k - y_k$, a useful interview result.

7. Practical Implementation

```
import torch
from torch import nn

logits = torch.tensor([[2.0, -1.0], [-0.5, 1.0]], requires_grad=True)
target = torch.tensor([0, 1])
loss = nn.CrossEntropyLoss()(logits, target) # logits + integer class labels
loss.backward()
print(loss.item(), logits.grad)
```

8. Code Explanation

`CrossEntropyLoss` combines log-softmax and negative log-likelihood stably. `target` contains class indices, not one-hot vectors. `backward()` stores gradients on `logits`, which would continue through model parameters in a real model.

9. Training / Evaluation

Fit transformations on training data only. Optimize BCE/CE for probabilities but evaluate classification with F1, PR-AUC, ROC-AUC, calibration, or cost-sensitive metrics as appropriate. A widening train/validation loss gap indicates overfitting; use regularization, more data, augmentation, or early stopping.

10. Complexity and Cost

Loss computation is $O(B)$ for scalar regression and $O(BK)$ for K -class softmax. It is usually negligible versus model forward/backward cost; dense vocabulary softmax can dominate LLM training.

11. Common Use Cases

MSE for numeric forecasting, BCE for binary risk/spam, cross-entropy for image/text classes and next-token prediction, Huber for noisy regression, and focal loss for severe detection imbalance.

12. Common Mistakes

Using MSE for class IDs, applying softmax before `CrossEntropyLoss`, mixing logits and probabilities, averaging with the wrong denominator, ignoring class imbalance, selecting thresholds on the test set, and interpreting a lower loss as automatically better business performance.

13. Edge Cases / Limitations

MSE overreacts to outliers; MAE may optimize slowly; CE can be overconfident; accuracy-aligned objectives may be non-differentiable. Loss weights can distort calibration and must reflect real error costs.

14. Variations

- **Weighted BCE/CE:** upweights rare/costly classes; placement-important for imbalance.
- **Focal loss:** downweights easy examples using $(1-p_t)^\gamma$; common in object detection.
- **Label smoothing:** replaces one-hot targets with softer targets; improves calibration/generalization in deep classifiers.
- **Contrastive/triplet losses:** learn embeddings by relative similarity; relevant to retrieval and face recognition.

15. Related Topics

Maximum likelihood yields cross-entropy; regularization changes the objective; gradients of the loss power SGD/Adam; calibration and thresholding convert probability outputs into decisions.

16. Interview Questions

1. **Loss vs metric?** Loss is differentiable training feedback; a metric measures task success.
2. **Why CE over MSE for classification?** It models likelihood and supplies stronger useful gradients for wrong confident predictions.
3. **What enters CrossEntropyLoss?** Raw logits and integer labels.
4. **Why is MSE outlier-sensitive?** Squaring makes large residuals dominate.
5. **When choose MAE?** When median-like robustness matters and outliers are common.
6. **What is Huber's benefit?** Quadratic near zero, linear for large errors.
7. **Why can BCE become infinite?** $\log(0)$; implementations use stable logits formulations.
8. **What does label smoothing do?** Prevents targets of exact probability one and reduces overconfidence.
9. **Does lower train loss guarantee better model?** No; check validation and task metric.
10. **How handle imbalance?** Weights, resampling, focal loss, and appropriate PR/F1/cost metrics.

17. Practice Tasks

Implement MSE and BCE with NumPy; compare MSE/MAE/Huber after injecting outliers; train a weighted BCE classifier on an imbalanced dataset; deliberately pass probabilities to `CrossEntropyLoss` and explain the failure.

18. Project Ideas

- **Delivery ETA predictor:** Huber regression, tabular delivery data, PyTorch/sklearn; demonstrates robust error modelling.
- **Fraud scorer:** weighted BCE, credit-card fraud data, PyTorch; demonstrates imbalance and threshold costs.
- **Product image retrieval:** contrastive loss, Fashion-MNIST, PyTorch; demonstrates representation learning.

19. Quick Revision

Key idea: define the error signal. Main formulas: MSE and CE above. Use CE for probabilistic classification and robust regression losses for outliers. Trap: logits/probabilities API mismatch. One-liner: a loss is the differentiable objective that tells learning which predictions to correct.

20. Final Cheat Sheet

Item	Notes
Definition	Differentiable penalty from prediction and target
Input/output	predictions/logits + labels -> scalar loss
Hyperparameters	class weights, delta , label smoothing
Metrics	task-dependent: MAE/RMSE, F1, PR-AUC, calibration
Pros/cons	trainable signal; wrong choice optimizes the wrong behavior
Best use	choose from target distribution and real error cost

Gradient Descent

1. Overview

Gradient descent is the general iterative method for minimizing differentiable objectives. Neural-network training uses it because exact closed-form solutions are unavailable for most architectures.

2. Intuition

Imagine descending a foggy hill: the local slope points uphill, so step in the opposite direction. Repeating small downhill steps reaches a low valley.

3. Prerequisites

Partial derivatives, vectors, chain rule, loss functions, and basic linear algebra.

4. Core Concepts

- **Gradient:** $\text{grad } J(\theta)$ points toward steepest local increase. Its negative is the descent direction.
- **Step size:** learning rate controls distance moved; too large diverges, too small is slow.
- **Local landscape:** convex losses have a global minimum; neural losses are non-convex but useful minima are often found.
- **Backpropagation:** efficiently applies the chain rule to obtain gradients in neural networks. Interview: it computes gradients; an optimizer applies updates.

5. Algorithm / Working Process

Initialize parameters; compute objective and gradient; update parameters; repeat until a budget, small gradient, or validation stopping condition. During inference, freeze parameters and run only the forward pass.

6. Mathematical Foundation

The update is $\theta_{t+1} = \theta_t - \eta \text{grad } J(\theta_t)$. A first-order expansion gives $J(\theta - \eta g) \approx J(\theta) - \eta \|g\|^2$, so sufficiently small η lowers the objective. For $J(\theta) = \theta^2$, gradient is 2θ , thus $\theta \leftarrow \theta(1 - 2\eta)$.

7. Practical Implementation

```
import numpy as np
x, y, w = np.array([1., 2., 3.]), np.array([2., 4., 6.]), 0.0
for _ in range(100):
    grad = (2 / len(x)) * np.sum((w * x - y) * x)
    w -= 0.1 * grad
assert abs(w - 2) < 1e-5
```

8. Code Explanation

The model is $\hat{y} = w * x$; the loop uses the MSE derivative and updates one scalar parameter. The assertion is a runnable check that the descent direction and scale are correct.

9. Training / Evaluation

Standardize numerical features so one coordinate does not dominate curvature. Plot training and validation loss; tune learning rate first. Underfitting may need more capacity/training; overfitting needs regularization or earlier stopping.

10. Complexity and Cost

One full gradient costs one forward/backward pass over all examples: roughly $O(nd)$ for linear models, or model-dependent for networks. Memory includes parameters, activations, and gradient buffers.

11. Common Use Cases

Linear/logistic regression, neural networks, matrix factorization, differentiable simulation, and fine-tuning foundation models.

12. Common Mistakes

Wrong gradient sign, no feature scaling, learning rate chosen from test results, forgetting to zero accumulated PyTorch gradients, and assuming convergence means a global optimum.

13. Edge Cases / Limitations

It can be slow in ill-conditioned ravines, stuck near saddle points, or diverge with poor scaling/learning rate. It cannot directly optimize discrete or non-differentiable metrics.

14. Variations

- **Line search:** chooses step sizes analytically/numerically; useful in classical optimization.
- **Newton method:** uses Hessian curvature; fast near a solution but costly ($O(d^2)$ memory / solve cost).
- **Coordinate descent:** updates coordinates separately; useful for some sparse convex objectives.

15. Related Topics

Batch, stochastic, and mini-batch GD differ only in how they estimate the gradient. Momentum, RMSProp, and Adam modify the update rule.

16. Interview Questions

1. **What is a gradient?** Vector of partial derivatives; steepest local increase.
2. **Why subtract it?** To lower the objective locally.
3. **What controls convergence?** Learning rate, curvature, gradient noise, initialization, and conditioning.
4. **Convex vs non-convex?** Convex has no bad local minima; deep losses need not be convex.
5. **Backprop vs GD?** Backprop computes gradients; GD uses them.
6. **Why scale features?** It improves conditioning and enables a common step size.
7. **What is a saddle point?** Gradient near zero but not a minimum.
8. **What signals divergence?** Exploding/NaN loss or increasing oscillations.
9. **Can GD optimize accuracy directly?** Usually no, because thresholded accuracy is non-differentiable.
10. **Stopping criteria?** Epoch budget plus validation early stopping; optionally gradient/objective tolerance.

17. Practice Tasks

Derive the MSE gradient; visualize descent on x^2 ; compare normalized versus unnormalized features; implement early stopping using a validation-loss patience counter.

18. Project Ideas

- **From-scratch linear regression:** NumPy, California Housing; demonstrates derivation and convergence plots.
- **Neural digit classifier:** PyTorch, MNIST; demonstrates backprop and LR tuning.
- **Optimizer dashboard:** NumPy/Matplotlib toy surfaces; demonstrates optimizer behavior visually.

19. Quick Revision

Key idea: repeatedly move opposite the slope. Formula: $\theta \leftarrow \theta - \eta \text{ grad } J$. Use for differentiable losses. Trap: a good direction still fails with a bad step size. One-liner: GD turns a derivative into a parameter update.

20. Final Cheat Sheet

Item	Notes
Input/output	parameters + gradient $\rightarrow$ updated parameters
Main step	subtract learning-rate-scaled gradient
Key hyperparameter	learning rate η

Item	Notes
Metrics	validation loss and task metric
Pros/cons	simple/general; sensitive to conditioning and step size
Best use	differentiable objectives

Batch Gradient Descent

1. Overview

Batch gradient descent computes each update using the entire training set. It is deterministic for fixed data and parameters, and is most practical for small datasets or convex objectives.

2. Intuition

Before each step, ask every training example which way to move, then average their opinions. The direction is stable but expensive to obtain.

3. Prerequisites

Gradient descent, vectorized arrays, average loss, and memory basics.

4. Core Concepts

- **Full gradient:** $g = (1/n) \sum_i \text{grad } \ell_i$; exact for the empirical training objective.
- **Epoch/update:** one batch-GD update consumes one full epoch; unlike mini-batch training, updates are infrequent.
- **Determinism:** no shuffle noise in the gradient; useful for debugging and theoretical convergence.
- **Vectorization:** compute all rows together rather than Python loops; interview: vectorization is faster but may increase memory pressure.

5. Algorithm / Working Process

Load all training features/labels, forward-pass all rows, average the loss, backpropagate once, update once, and repeat. Validate after each update/epoch. Inference is unchanged from the underlying model.

6. Mathematical Foundation

$J(\theta) = (1/n) \sum_i \ell_i(\theta)$ and $\theta \leftarrow \theta - \eta (1/n) \sum_i \text{grad } \ell_i(\theta)$. For a convex L -smooth objective, a sufficiently small fixed rate (commonly $\eta < 2/L$) decreases the objective.

7. Practical Implementation

```
import torch
X = torch.tensor([[1.], [2.], [3.]])
y = 2 * X
model = torch.nn.Linear(1, 1)
opt = torch.optim.SGD(model.parameters(), lr=.1)
```

```
for _ in range(200):  
    opt.zero_grad(); loss = torch.nn.functional.mse_loss(model(X), y)  
    loss.backward(); opt.step() # one update after all three examples
```

8. Code Explanation

There is no data loader loop: the complete dataset is passed to the model before `step()`. `mse_loss` defaults to a mean, so gradients are averaged rather than summed.

9. Training / Evaluation

Use when the whole dataset fits comfortably in memory. Evaluate validation loss regularly; because updates are sparse, an epoch-level curve has few points. Scale features and consider a line search or learning-rate schedule for convex problems.

10. Complexity and Cost

Each update costs $O(nd)$ for a d -feature linear model and generally requires holding a full forward graph/activations. It has low gradient variance but high latency per update and poor scalability.

11. Common Use Cases

Small tabular regression/classification, teaching, deterministic baselines, and some classical convex solvers.

12. Common Mistakes

Calling it batch GD when using batches, using a sum loss that changes effective LR with dataset size, loading datasets too large for memory, and judging progress only by update count rather than data processed.

13. Edge Cases / Limitations

Large datasets make one update slow and memory-heavy; it cannot exploit streaming data naturally. The exact full-data direction can still lead poorly conditioned objectives to zig-zag.

14. Variations

- **Full-batch L-BFGS:** curvature-informed method for small smooth models; useful in research/classical ML.
- **Distributed synchronous full batch:** aggregates workers' gradients; expensive communication but deterministic aggregation.
- **Gradient accumulation:** simulates a larger batch when memory is limited; placement-relevant for deep learning.

15. Related Topics

It is the zero-gradient-noise endpoint of the batch-size spectrum; SGD and mini-batch GD trade exactness for cheaper, more frequent updates.

16. Interview Questions

1. **What data does one update use?** Every training example.
2. **Main advantage?** Exact, stable empirical gradient.
3. **Main drawback?** Slow, memory-heavy updates.
4. **Updates per epoch?** One.
5. **Why not for huge data?** Full forward/backward pass per step is too costly.
6. **Is it always faster to converge?** Fewer noisy steps, but often much slower wall-clock progress.
7. **Does shuffling matter?** Not to the exact full gradient, barring numerical/distributed details.
8. **Mean vs sum reduction?** Sum scales gradients with n ; mean does not.
9. **Does it avoid local minima?** No.
10. **When use it?** Small data, reproducible baselines, some convex optimization.

17. Practice Tasks

Implement full-batch linear regression in NumPy; change mean loss to sum and retune LR; measure time/memory as n grows; compare its loss curve with mini-batches.

18. Project Ideas

- **Housing-price baseline:** full-batch linear regression, California Housing, NumPy.
- **Convex logistic benchmark:** sklearn/PyTorch, Breast Cancer data; compare reproducible runs.
- **Optimizer visualizer:** toy two-dimensional loss; show one long, stable update trajectory.

19. Quick Revision

Key idea: exact average gradient. Formula: $\mathbf{g} = (1/n) \sum \text{grad } \ell_i$. Use for small datasets. Trap: it has one update per epoch. One-liner: batch GD is stable but waits for all data before moving.

20. Final Cheat Sheet

Item	Notes
Input/output	full dataset + parameters $\rightarrow$ one update
Hyperparameter	learning rate
Cost	$O(n)$ examples per update; high memory/latency
Pros/cons	low noise; poor large-scale scalability
Best use	small datasets and deterministic baselines

Stochastic Gradient Descent

1. Overview

Stochastic gradient descent (SGD) updates parameters using one sampled training example at a time. Its gradient is noisy but inexpensive and, in expectation, points toward the full gradient.

2. Intuition

Instead of polling every voter before moving downhill, ask one randomly chosen voter. Individual advice is noisy, but many fast corrections reach a good valley.

3. Prerequisites

Gradient descent, random sampling, expectation, epochs, and feature scaling.

4. Core Concepts

- **Stochastic gradient:** $g_i = \text{grad } \ell_i(\theta)$ estimates the full gradient. With uniform sampling, $E[g_i] = \text{grad } J$.
- **Noise:** can escape shallow saddles and helps throughput, but produces a jagged loss curve.
- **Shuffling:** randomize rows each epoch to avoid order-dependent cycles and biased sequences.
- **Learning-rate decay:** a decreasing rate helps noisy updates settle near a minimum. Interview: constant LR often reaches a neighborhood, not exact convergence.

5. Algorithm / Working Process

Shuffle data; for every (x_i, y_i) , forward-pass one example, calculate loss/gradient, update immediately, and repeat for epochs. Infer with a normal forward pass after training.

6. Mathematical Foundation

For $J = (1/n) \sum \ell_i$, choose i uniformly and update $\theta_{t+1} = \theta_t - \eta_t \text{grad } \ell_i(\theta_t)$. Unbiasedness follows because $E_i[\text{grad } \ell_i] = (1/n) \sum_i \text{grad } \ell_i$. Its variance is much larger than a full gradient's variance.

7. Practical Implementation

```
import torch
from torch.utils.data import DataLoader, TensorDataset
X = torch.tensor([[1.], [2.], [3.]])
y = 2 * X
loader = DataLoader(TensorDataset(X, y), batch_size=1, shuffle=True)
model = torch.nn.Linear(1, 1)
opt = torch.optim.SGD(model.parameters(), lr=.05)
for _ in range(100):
    for xb, yb in loader:
        opt.zero_grad()
        torch.nn.functional.mse_loss(model(xb), yb).backward()
    opt.step()
```

8. Code Explanation

`batch_size=1` makes this true SGD. Each row triggers its own `zero_grad`, backward pass, and update; shuffle changes row order every epoch.

9. Training / Evaluation

Use a validation split and smooth the displayed training loss with a moving average. Tune LR and decay; standardize inputs. If validation metrics oscillate, lower LR, add momentum, or move to mini-batches.

10. Complexity and Cost

One update costs $O(d)$ for a linear model and uses very little data memory, but frequent tiny operations can underuse GPUs. One epoch costs $O(nd)$; optimizer state is minimal for vanilla SGD.

11. Common Use Cases

Online learning, streaming data, sparse linear models, large datasets on CPU, and as the conceptual basis for mini-batch neural training.

12. Common Mistakes

Not shuffling, using one enormous fixed LR, treating noisy per-example loss as validation performance, failing to scale features, and using batch size one on a GPU when throughput matters.

13. Edge Cases / Limitations

Noisy gradients can be unstable with outliers or poorly scaled inputs. Sequential correlated samples can bias updates. It usually converges more slowly in objective value than well-tuned mini-batch methods.

14. Variations

- **Online SGD:** learn continuously as examples arrive; relevant to recommender/fraud streams.
- **SGD with momentum:** smooths directions; essential deep-learning variant.
- **Averaged SGD:** averages late iterates for better convex-objective stability; theoretical/classical relevance.

15. Related Topics

Mini-batch GD averages several stochastic gradients to reduce variance. Momentum, RMSProp, and Adam change how SGD directions are scaled and remembered.

16. Interview Questions

1. **What is SGD's batch size?** One example.
2. **Is its gradient exact?** No, but it is unbiased under uniform sampling.
3. **Why shuffle?** To break harmful order/correlation effects.
4. **Why noisy?** Each example's gradient differs from the dataset average.
5. **Benefit of noise?** Cheap updates and possible escape from saddles/sharp regions.
6. **Why decay LR?** To reduce late-stage wandering.
7. **SGD vs mini-batch?** One sample versus several; mini-batch is better hardware/variance trade-off.
8. **Good for streaming?** Yes; it needs only one example at a time.
9. **Why poor GPU use?** Tiny kernels and transfer overhead.
10. **How reduce instability?** Scale data, lower LR, shuffle, add momentum, or batch examples.

17. Practice Tasks

Implement SGD linear regression; compare shuffled versus sorted targets; plot raw and moving-average loss; add an inverse-time LR schedule and compare final validation loss.

18. Project Ideas

- **Streaming price predictor:** incremental SGD, synthetic event stream, sklearn.
- **News-topic classifier:** sparse text + SGDClassifier, 20 Newsgroups; demonstrates scalability.
- **Online anomaly detector:** one-at-a-time updates, transaction stream; demonstrates monitoring.

19. Quick Revision

Key idea: update after one sample. Formula: $\theta \leftarrow \theta - \eta \text{grad } \ell_i$. Use for streaming or very large data. Trap: noisy training curves are normal; validation drives decisions. One-liner: SGD trades exact gradients for fast, cheap feedback.

20. Final Cheat Sheet

Item	Notes
Input/output	one sampled example -> parameter update
Hyperparameters	LR, schedule, shuffle seed
Cost	low memory, $O(d)$ per update
Pros/cons	scalable/online; high variance and weak GPU efficiency
Best use	streams and sparse classical ML

Mini-batch Gradient Descent

1. Overview

Mini-batch gradient descent uses a small batch of B examples per update. It is the standard training method for deep learning because it balances SGD's frequent feedback with batch GD's stable directions and accelerator efficiency.

2. Intuition

Ask a small committee rather than one voter or the whole city. Their average is less erratic than one opinion and arrives much faster than a census.

3. Prerequisites

SGD, batching, tensors, data loaders, and GPU memory concepts.

4. Core Concepts

- **Batch gradient estimate:** $\mathbf{g}_B = (1/B) \sum_{i \text{ in batch}} \text{grad } \ell_i$; variance generally falls as B grows.
- **Epoch:** one pass through data; updates per epoch are $\text{ceil}(n/B)$.
- **Batch size trade-off:** larger batches improve device utilization but use more memory and have fewer updates per epoch.

- **Gradient accumulation:** sum/average gradients across several micro-batches before stepping to emulate a larger effective batch.

5. Algorithm / Working Process

Shuffle and partition training data into batches. For each: forward pass, mean loss, backward pass, update. Accumulate validation metrics without gradients. Inference commonly uses any convenient batch size permitted by memory.

6. Mathematical Foundation

$\theta_{t+1} = \theta_t - \eta \frac{1}{B} \sum_{i \in S_t} \text{grad}_{\theta} \ell_i(\theta_t)$. Sampling without replacement within an epoch gives a near-unbiased estimator; its variance is lower than one-example SGD. Linear LR scaling (η proportional to B) is only a starting heuristic, not a guarantee.

7. Practical Implementation

```
from torch.utils.data import DataLoader, TensorDataset
import torch
X, y = torch.randn(256, 10), torch.randint(0, 2, (256,))
loader = DataLoader(TensorDataset(X, y), batch_size=32, shuffle=True)
model = torch.nn.Linear(10, 2); opt = torch.optim.SGD(model.parameters(), lr=.1)
for xb, yb in loader:
    opt.zero_grad(); loss = torch.nn.functional.cross_entropy(model(xb), yb)
    loss.backward(); opt.step()
```

8. Code Explanation

The data loader provides 32 examples, `cross_entropy` averages their loss, and each `step()` updates from that averaged batch gradient. In actual training, wrap this loop in epochs and a validation loop.

9. Training / Evaluation

Start with a batch size that fits memory (often 16–256 depending on model) and tune LR jointly. Use a fixed validation set, record examples/steps as well as epochs, and use mixed precision or accumulation when GPU memory limits the desired effective batch.

10. Complexity and Cost

Per update is roughly $O(Bd)$ for linear models; an epoch stays $O(nd)$. Memory grows with B because activations are retained for backprop. Batches exploit GPU matrix operations well.

11. Common Use Cases

CNNs, transformers, LLM fine-tuning, tabular neural networks, and almost all modern GPU training pipelines.

12. Common Mistakes

Calling `step()` after every micro-batch while claiming accumulation, not dividing accumulated gradients, choosing batch size without LR retuning, reporting batch-averaged loss incorrectly, and using validation data in the loader shuffle/training loop.

13. Edge Cases / Limitations

Very large batches can hurt generalization or require LR warm-up; very small batches make BatchNorm statistics noisy and waste accelerators. Batch size may be constrained by sequence length, not just row count.

14. Variations

- **Micro-batching + accumulation:** memory-safe effective large batches; important for LLMs.
- **Distributed data parallel:** each worker processes a mini-batch then all-reduces gradients; production-scale training.
- **Dynamic batching:** groups similar-length sequences to reduce padding; useful for NLP.

15. Related Topics

It is a middle ground between batch GD and SGD. Momentum/Adam operate on mini-batch gradients; BatchNorm behavior depends on mini-batch statistics.

16. Interview Questions

1. **Why is mini-batch default?** It balances variance, update frequency, and GPU throughput.
2. **Updates per epoch?** $\text{ceil}(n/B)$.
3. **What grows with batch size?** Activation memory and compute per update.
4. **What falls with batch size?** Gradient variance and updates per epoch.
5. **Can bigger batches need bigger LR?** Often, but retune and warm up.
6. **What is effective batch size?** $\text{micro_batch} * \text{accumulation_steps} * \text{workers}$ when gradients are averaged.
7. **Why shuffle?** Better representative batches and less order bias.
8. **Does inference need same batch size?** No.
9. **Why does BatchNorm dislike tiny batches?** Its mean/variance estimates become noisy.
10. **How handle OOM?** Reduce batch, use accumulation/mixed precision, or shorten sequences.

17. Practice Tasks

Train the same classifier at batches 1, 32, and 256; plot step and epoch loss; implement accumulation over four micro-batches; measure peak GPU memory.

18. Project Ideas

- **CIFAR-10 classifier:** PyTorch/torchvision; compare batch sizes and throughput.
- **Text sentiment model:** Hugging Face + IMDB; use dynamic padding and accumulation.
- **Batch-size tuner:** benchmark script for a chosen GPU; useful MLOps/resume artifact.

19. Quick Revision

Key idea: average a small subset per update. Formula: $g_B = (1/B) \sum \text{grad } \ell_i$. Use for most neural training. Trap: batch size and LR must be tuned together. One-liner: mini-batches make gradient descent practical on accelerators.

20. Final Cheat Sheet

Item	Notes
Input/output	B examples -> one averaged-gradient update
Hyperparameters	batch size, LR, accumulation steps
Cost	epoch $O(nd)$; activation memory $O(B)$
Pros/cons	stable and GPU-friendly; memory/LR-sensitive
Best use	modern deep learning

Learning Rate

1. Overview

The learning rate (LR) is the optimizer's step-size hyperparameter. It is often the most important training choice: it determines whether loss descends quickly, crawls, oscillates, or diverges.

2. Intuition

On a hill, giant steps overshoot the valley, tiny steps take forever, and well-sized steps descend quickly without bouncing.

3. Prerequisites

Gradient descent, loss curves, validation, epochs/steps, and feature normalization.

4. Core Concepts

- **Magnitude:** LR multiplies the update direction. Same optimizer, wrong LR, radically different result.
- **Schedules:** change LR over training; high early LR explores, lower late LR refines.
- **Warm-up:** gradually increase LR from a small value; stabilizes early transformer/large-batch training.
- **LR range test:** briefly sweep exponentially increasing rates and choose a stable, rapidly improving region.

5. Algorithm / Working Process

Choose a base LR; optionally warm up; update parameters using current LR; decay it by schedule or when validation stalls; select final configuration using validation, never test performance.

6. Mathematical Foundation

Basic update: $\theta_{t+1} = \theta_t - \eta_t g_t$. Common schedules: step decay $\eta_t = \eta_0 \gamma^{\lfloor t/s \rfloor}$, exponential $\eta_t = \eta_0 e^{-kt}$, cosine $\eta_t = \eta_{\min} + 0.5(\eta_0 - \eta_{\min})(1 + \cos(\pi t/T))$. For an L -smooth objective, stable fixed-step GD conventionally needs $\eta < 2/L$.

7. Practical Implementation

```
import torch
model = torch.nn.Linear(10, 2)
opt = torch.optim.AdamW(model.parameters(), lr=3e-4)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=20, eta_min=1e-6)
for epoch in range(20):
    # run mini-batch training here, calling opt.step() for each batch
    scheduler.step()
    print(epoch, scheduler.get_last_lr()[0])
```

8. Code Explanation

The optimizer reads its LR from parameter groups. The cosine scheduler changes it after each epoch here; if configured for step-level scheduling, call it after each update instead. `T_max` must match that chosen unit.

9. Training / Evaluation

Plot LR and train/validation loss together. Loss exploding or NaN usually means LR is too high (also check data/numerics). Slow, nearly flat improvement may mean too low. Tune LR with batch size, optimizer, regularization, and schedule.

10. Complexity and Cost

An LR schedule adds negligible compute and memory. Its operational cost is experiment runs; logging it prevents opaque training failures.

11. Common Use Cases

Every gradient-based model; warm-up/cosine decay in transformers; reduce-on-plateau in smaller supervised models; fine-tuning with smaller rates than pretraining.

12. Common Mistakes

Using a copied LR without matching batch/optimizer, stepping scheduler at the wrong frequency, using test loss for `ReduceLROnPlateau`, forgetting warm-up in unstable large-model runs, and changing many hyperparameters at once.

13. Edge Cases / Limitations

No universal LR exists. Adaptive optimizers still require LR tuning. Schedules cannot correct broken labels, unnormalized inputs, exploding gradients, or a fundamentally unsuitable model.

14. Variations

- **Constant LR:** simple baseline; suitable for short stable training.
- **Reduce on plateau:** decay after validation metric stalls; practical for tabular/CV training.
- **Cosine with warm restarts:** periodically raises LR; useful when training longer or exploring basins.
- **One-cycle:** rises then falls once; placement-relevant practical schedule.

15. Related Topics

LR controls every GD variant. Momentum can tolerate/benefit from different rates; Adam/RMSProp normalize directions but retain a global base LR; gradient clipping handles magnitude spikes separately.

16. Interview Questions

1. **What does LR control?** Update magnitude.
2. **Too high?** Oscillation, exploding loss, divergence/NaNs.
3. **Too low?** Very slow progress or apparent stagnation.
4. **Why decay?** Smaller late updates settle in a minimum.
5. **Why warm-up?** Avoid unstable early large updates.
6. **Does Adam remove LR tuning?** No.
7. **Why LR differs in fine-tuning?** Pretrained weights need gentler changes.
8. **Scheduler per epoch or batch?** Depends on scheduler definition; be consistent.
9. **What does plateau scheduling monitor?** Validation metric/loss.
10. **How choose initial LR?** Known baseline or LR range test, then validate.

17. Practice Tasks

Run a log-spaced LR sweep; plot divergence/slow/stable curves; add warm-up to a transformer fine-tune; compare constant, step, and cosine schedules at equal training budgets.

18. Project Ideas

- **LR finder notebook:** PyTorch + any dataset; demonstrates diagnostic experimentation.
- **CIFAR schedule study:** ResNet + CIFAR-10; compare schedules fairly.
- **Fine-tuning tracker:** Hugging Face + AG News; log LR, loss, F1, and checkpoints.

19. Quick Revision

Key idea: size of each move. Formula: $\theta_{t+1} = \theta_t - \eta_t g_t$. Use a schedule when training is long/nonstationary. Trap: scheduler units and batch-size changes alter the effective plan. One-liner: LR decides whether the optimizer learns, explodes, or wastes time.

20. Final Cheat Sheet

Item	Notes
Definition	scalar multiplier of update direction
Key hyperparameters	base LR, warm-up, decay schedule

Item	Notes
Metrics	validation loss/task metric; train curve stability
Pros/cons	no added compute; highly sensitive
Best use	tune first for any optimizer

Momentum

1. Overview

Momentum augments gradient descent with a running velocity. It accelerates movement in consistent directions and damps oscillations across steep, narrow curvature; it is widely used with SGD for vision models.

2. Intuition

A heavy ball rolling downhill keeps useful speed in the forward direction, while side-to-side bumps cancel out instead of causing constant zig-zags.

3. Prerequisites

Gradient descent, exponential moving averages, learning rate, and vector gradients.

4. Core Concepts

- **Velocity:** an exponential average of past gradients; preserves directional signal.
- **Momentum coefficient μ :** often around 0.9 ; closer to one means longer memory.
- **Damping:** conflicting gradient components cancel in velocity, reducing ravine oscillation.
- **Nesterov momentum:** measures gradient after a look-ahead move; common interview extension.

5. Algorithm / Working Process

At each mini-batch, compute gradient g ; update velocity from old velocity and g ; update parameters from velocity. Initialize $v=0$; during inference velocity is irrelevant.

6. Mathematical Foundation

Classical form: $v_t = \mu v_{(t-1)} - \eta g_t$, $\theta_t = \theta_{(t-1)} + v_t$. Equivalent gradient-average form: $m_t = \mu m_{(t-1)} + (1-\mu)g_t$, $\theta_t \leftarrow \theta_t - \eta m_t$. Nesterov approximately computes $g_t = \text{grad } J(\theta_{(t-1)} + \mu v_{(t-1)})$ before the update.

7. Practical Implementation

```
import torch
model = torch.nn.Linear(10, 2)
optimizer = torch.optim.SGD(model.parameters(), lr=0.05, momentum=0.9,
```

```
nesterov=True)  
# Each training batch: optimizer.zero_grad(); loss.backward(); optimizer.step()
```

8. Code Explanation

PyTorch stores one velocity buffer per parameter. `momentum=.9` retains most prior direction; `nesterov=True` uses the look-ahead variant and requires nonzero momentum.

9. Training / Evaluation

Tune LR and momentum together; adding momentum usually changes the stable LR range. Track validation metrics and inspect loss for overshoot. Weight decay, LR schedule, normalized data, and gradient clipping remain separate choices.

10. Complexity and Cost

One extra velocity tensor per parameter: approximately one additional parameter-size memory buffer. Compute is a few elementwise operations, negligible next to backprop.

11. Common Use Cases

CNN/ResNet image training, large-scale supervised learning with SGD, and smooth differentiable objectives where generalization of SGD is desired.

12. Common Mistakes

Using momentum with an unretuned LR, confusing velocity with a second derivative, setting `mu` extremely close to one without care, enabling Nesterov incorrectly, and using it to mask exploding gradients.

13. Edge Cases / Limitations

Stored velocity can carry the optimizer past a changing optimum; abrupt distribution shifts and very noisy gradients may need lower momentum. It does not adapt steps per parameter like RMSProp/Adam.

14. Variations

- **Classical momentum:** gradient at current position; essential placement topic.
- **Nesterov accelerated gradient:** look-ahead correction; common in CV recipes/interviews.
- **Dampened momentum:** reduces new-gradient contribution; occasionally useful for stability.

15. Related Topics

Momentum is a first-moment-like memory. Adam includes momentum plus adaptive second-moment scaling; RMSProp supplies only the latter. LR scheduling controls the scale of momentum updates.

16. Interview Questions

1. **Why momentum?** Faster consistent progress and less oscillation.
2. **What does `mu` mean?** Memory/decay of past velocity.

3. **Typical value?** Around 0.9, but tune it.
4. **Does it add parameters?** No model parameters, only optimizer state.
5. **Momentum vs Adam?** Momentum has one global scaling; Adam also adapts per parameter.
6. **What is Nesterov?** Compute gradient at a look-ahead position.
7. **Why helps ravines?** Averages alternating steep-direction gradients while accumulating valley direction.
8. **Can it diverge?** Yes, with too high LR/momentum.
9. **Is it used at inference?** No.
10. **What must be retuned after enabling it?** Learning rate and often schedule.

17. Practice Tasks

Implement momentum on a 2D quadratic; compare trajectories with plain GD; sweep μ in $\{0, .5, .9, .99\}$; compare classical and Nesterov SGD on MNIST.

18. Project Ideas

- **Optimizer surface lab:** NumPy/Matplotlib; visualize ravine trajectories.
- **CIFAR-10 ResNet:** PyTorch SGD+Nesterov; demonstrate production-standard CV training.
- **Momentum ablation report:** one architecture/dataset, controlled LR tuning; demonstrates experimental discipline.

19. Quick Revision

Key idea: remember direction. Formula: $\mathbf{v} = \mu \mathbf{v} - \eta \mathbf{g}$; $\boldsymbol{\theta} += \mathbf{v}$. Use when SGD zig-zags. Trap: it needs LR retuning. One-liner: momentum turns repeated agreement among gradients into speed.

20. Final Cheat Sheet

Item	Notes
Input/output	gradient + velocity -> updated parameters
Hyperparameters	LR, momentum μ , Nesterov flag
Cost	one extra state tensor/parameter
Pros/cons	fast/smooth; not coordinate-adaptive
Best use	SGD-based deep vision and stable supervised training

Adam Optimizer

1. Overview

Adam (Adaptive Moment Estimation) combines momentum-like first-moment averaging with RMSProp-like adaptive scaling from squared gradients. It is a strong, low-tuning baseline for transformers, NLP, sparse gradients, and many deep-learning tasks.

2. Intuition

Adam remembers both where gradients usually point and how volatile each parameter's gradients are. Parameters with consistently large gradients receive relatively smaller normalized steps.

3. Prerequisites

SGD, momentum, exponential moving averages, learning rate, and elementwise vector operations.

4. Core Concepts

- **First moment m** : smoothed gradient direction, analogous to momentum.
- **Second moment v** : smoothed squared gradient magnitude; scales coordinates adaptively.
- **Bias correction**: EMAs start at zero, so early estimates are biased low; correction matters in initial steps.
- **AdamW**: decouples weight decay from adaptive update; usually preferred for transformer/deep model regularization.

5. Algorithm / Working Process

Initialize $m=v=0$ and step $t=0$. For every mini-batch calculate g , update moment estimates, bias-correct them, divide the direction by root-mean-square magnitude plus epsilon, then update parameters. Inference has no optimizer steps.

6. Mathematical Foundation

$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$; $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ (elementwise). Bias correction: $\hat{m}_t = m_t / (1 - \beta_1^t)$, $\hat{v}_t = v_t / (1 - \beta_2^t)$. Update: $\theta \leftarrow \theta - \eta \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$. Typical defaults are $\beta_1=.9$, $\beta_2=.999$, $\epsilon=1e-8$.

7. Practical Implementation

```
import torch
model = torch.nn.Sequential(torch.nn.Linear(20, 64), torch.nn.ReLU(),
                             torch.nn.Linear(64, 2))
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=1e-2)
# per mini-batch: optimizer.zero_grad(); loss.backward(); optimizer.step()
```

8. Code Explanation

AdamW maintains m and v buffers automatically. **weight_decay** is decoupled from the adaptive gradient calculation, unlike traditional L2 penalty behavior in Adam. Exclude biases and normalization parameters from decay when following common transformer recipes.

9. Training / Evaluation

Start with known task-scale defaults (often $1e-3$ for small networks, $1e-5$ – $5e-4$ for fine-tuning depending on model), then validate LR, weight decay, and schedule. Use warm-up for transformer training. Monitor validation metrics: Adam's faster training loss is not proof of better generalization than SGD.

10. Complexity and Cost

Adam stores $\mathbf{m}$ and $\mathbf{v}$, roughly two additional parameter-sized tensors (plus gradients), so optimizer memory is about 2x SGD's state and is material for large models. Elementwise compute overhead is small relative to backprop.

11. Common Use Cases

Transformer pretraining/fine-tuning, LLM adapters, NLP, GANs, sparse embeddings, and quick neural-network baselines.

12. Common Mistakes

Saying Adam has no LR, using Adam instead of AdamW while expecting decoupled weight decay, applying decay to all parameters blindly, omitting warm-up for an unstable transformer run, and comparing optimizers without separate LR tuning.

13. Edge Cases / Limitations

Adaptive normalization can generalize worse than tuned SGD on some vision tasks. Memory overhead is high for billion-parameter models. Tiny $\mathbf{v}$ or bad epsilon/LR choices can yield unstable effective steps; Adam cannot fix noisy labels or leakage.

14. Variations

- **AdamW**: decoupled weight decay; placement-essential and common default.
- **AMSGrad**: uses a nondecreasing second-moment maximum; theoretical stability extension.
- **AdaBelief/Lion**: alternative adaptive/update rules; research/project awareness, less essential.
- **8-bit Adam**: compresses optimizer states; relevant to memory-constrained LLM training.

15. Related Topics

Adam combines momentum's first moment with RMSProp's second-moment idea. AdamW handles regularization differently from L2-in-Adam. LR schedules, warm-up, clipping, and batch size remain critical.

16. Interview Questions

1. **What does Adam stand for?** Adaptive Moment Estimation.
2. **What are $\mathbf{m}$ and $\mathbf{v}$?** EMAs of gradients and squared gradients.
3. **Why bias correction?** Both EMA estimates begin at zero.
4. **Why square gradients?** To estimate coordinate-wise magnitude without sign cancellation.
5. **Why epsilon?** Numerical stability and bounded division.
6. **Adam vs SGD momentum?** Adam also divides by adaptive RMS scale.
7. **Adam vs AdamW?** AdamW decouples weight decay from gradient adaptation.
8. **Does Adam need a schedule?** Often yes, especially warm-up/decay for transformers.
9. **Why more memory?** Stores two moment tensors per parameter.
10. **When prefer SGD?** When tuned SGD gives better target generalization, often some vision tasks.

17. Practice Tasks

Implement Adam from NumPy equations; verify bias correction changes first ten steps; compare AdamW and SGD+momentum with tuned LR; measure optimizer state memory for a network.

18. Project Ideas

- **Sentiment fine-tuner:** DistilBERT + AdamW, IMDB; demonstrates warm-up and decay.
- **Sparse recommender:** embeddings + Adam, MovieLens; demonstrates sparse-gradient suitability.
- **Optimizer benchmark:** same MLP across datasets with controlled tuning; strong interview discussion artifact.

19. Quick Revision

Key idea: momentum plus per-parameter RMS scaling. Formula: $\theta \leftarrow \theta - \eta \cdot \frac{m}{\sqrt{v} + \epsilon}$. Use as a robust deep-learning baseline. Trap: AdamW and Adam's L2 behavior are not equivalent. One-liner: Adam adapts how far each parameter moves using its gradient history.

20. Final Cheat Sheet

Item	Notes
Input/output	gradient + m, v states -> adaptive update
Hyperparameters	LR, β_1 , β_2 , epsilon, weight decay
Cost	two optimizer-state tensors per parameter
Pros/cons	fast, low tuning; memory-heavy, may generalize worse than SGD
Best use	transformers, NLP, sparse/deep baselines; prefer AdamW

RMSProp

1. Overview

RMSProp adapts each parameter's step size by dividing its gradient by a moving root-mean-square of recent squared gradients. It was designed to improve AdaGrad's endlessly growing accumulator and is useful for non-stationary, noisy objectives.

2. Intuition

If a parameter's gradients are repeatedly huge, its local step is reduced; if gradients are consistently small, it can take a relatively larger step. Unlike AdaGrad, it gradually forgets stale history.

3. Prerequisites

Gradient descent, elementwise operations, squared gradients, and exponential moving averages.

4. Core Concepts

- **Squared-gradient cache:** tracks recent magnitude, not direction.

- **Decay ρ / α :** controls how quickly old magnitudes are forgotten; common value around $.9$.
- **Coordinate adaptation:** each parameter has its own effective scale.
- **AdaGrad contrast:** AdaGrad accumulates all squares and can shrink learning rates too far; RMSProp uses a moving average.

5. Algorithm / Working Process

For each mini-batch calculate g ; update the squared-gradient cache; divide g by the cache's square root plus epsilon; update parameters. Optionally combine with momentum. Nothing runs at inference.

6. Mathematical Foundation

$s_t = \rho * s_{(t-1)} + (1-\rho)g_t^2$; $\theta_{(t+1)} = \theta_t - \eta * g_t / (\sqrt{s_t} + \epsilon)$. All products, squares, divisions, and roots are elementwise. Effective coordinate LR is approximately $\eta / (\sqrt{s_t} + \epsilon)$.

7. Practical Implementation

```
import torch
model = torch.nn.Linear(10, 2)
optimizer = torch.optim.RMSprop(model.parameters(), lr=1e-3, alpha=.99, eps=1e-8,
momentum=.0)
# per mini-batch: optimizer.zero_grad(); loss.backward(); optimizer.step()
```

8. Code Explanation

α is the squared-gradient moving-average coefficient (the ρ in the formula). PyTorch may optionally add momentum; with $\text{momentum}=0$, this is the basic RMSProp update.

9. Training / Evaluation

Tune LR, α , epsilon, and optionally momentum on validation metrics. Normalize inputs and use mini-batches. Plot both loss and gradient norms: adaptive scaling helps, but high LR can still diverge.

10. Complexity and Cost

RMSProp stores one extra squared-gradient tensor per parameter, between SGD and Adam in memory cost. Compute overhead is elementwise and minor compared with backpropagation.

11. Common Use Cases

Historically recurrent neural networks, noisy/non-stationary deep-learning objectives, reinforcement learning baselines, and experiments where Adam is too state-heavy or different behavior is desired.

12. Common Mistakes

Confusing RMSProp with Adam, using an unvalidated default LR, assuming adaptation removes the need for scaling/schedules, letting epsilon be zero, and calling the cache a gradient average rather than a squared-

gradient average.

13. Edge Cases / Limitations

It has no first-moment direction memory unless momentum is added. It can be less robust/default-friendly than AdamW in modern transformer recipes, and adaptive scaling can still be destabilized by bad data or an excessive base LR.

14. Variations

- **Centered RMSProp**: estimates variance by subtracting squared mean gradient; available in PyTorch, occasionally useful.
- **RMSProp with momentum**: adds directional smoothing; relevant in RL/CNN recipes.
- **AdaGrad**: no forgetting; useful for sparse convex features but can decay too aggressively.
- **Adam**: RMSProp-style second moment plus first moment and bias correction; placement-essential comparison.

15. Related Topics

RMSProp addresses AdaGrad's decaying-rate problem. Adam adopts its moving squared-gradient normalization and adds momentum/bias correction. Learning-rate scheduling still changes the global scale.

16. Interview Questions

1. **What does RMSProp store?** EMA of squared gradients.
2. **Why square gradients?** Magnitudes should not cancel by sign.
3. **Why divide by RMS?** To shrink steps in consistently steep coordinates.
4. **RMSProp vs AdaGrad?** RMSProp forgets old squared gradients.
5. **RMSProp vs Adam?** Adam adds first moment and bias correction.
6. **What does ρ do?** Sets memory of the squared-gradient cache.
7. **Why epsilon?** Prevent division by zero/numerical instability.
8. **Can it use momentum?** Yes, as an extension.
9. **Does it remove LR tuning?** No.
10. **Where is it common?** RNN history and reinforcement-learning implementations.

17. Practice Tasks

Implement RMSProp from NumPy equations; contrast cache growth with AdaGrad; train an RNN/MLP using SGD, RMSProp, and Adam; sweep α and LR while logging validation loss.

18. Project Ideas

- **Character-level RNN**: PyTorch + Tiny Shakespeare; compare RMSProp and Adam.
- **CartPole agent**: Gymnasium + PyTorch; RMSProp policy/value baseline.
- **Adaptive optimizer study**: Fashion-MNIST MLP; report optimizer state, speed, and F1.

19. Quick Revision

Key idea: normalize by recent gradient magnitude. Formula: $s = \rho s + (1 - \rho) g^2$; $\theta \leftarrow \theta - \frac{\eta g}{(\sqrt{s} + \epsilon)}$. Use for noisy/non-stationary objectives. Trap: it remembers magnitude, not direction. One-liner: RMSProp prevents persistently steep coordinates from dominating updates.

20. Final Cheat Sheet

Item	Notes
Definition	per-parameter adaptive optimizer using squared-gradient EMA
Input/output	gradient + cache $\rightarrow$ scaled parameter update
Hyperparameters	LR, ρ / α , epsilon, optional momentum
Cost	one state tensor per parameter
Pros/cons	handles changing scales; lacks first moment by default
Best use	RNN/RL history and adaptive-optimizer comparisons

Placement Comparison Cheat Sheet

Method	Gradient per update	State per parameter	Main strength	Main risk	Typical use
Batch GD	all n rows	none	exact/stable direction	slow, memory-heavy updates	small convex data
SGD	1 row	none	online, cheap updates	high variance	streams/sparse ML
Mini-batch GD	B rows	none	GPU efficiency + stability	batch/LR coupling	default deep learning
Momentum	mini-batch	velocity	speeds consistent direction	overshoot	SGD vision training
RMSProp	mini-batch	squared cache	adapts coordinate scales	no direction memory	noisy/RNN/RL workloads
Adam/AdamW	mini-batch	first + second moments	robust default	memory/generalization trade-off	transformers/NLP

Interview summary: loss defines *what* is minimized; the gradient defines *which direction* reduces it; batch size determines *how accurately* the gradient is estimated; learning rate determines *how far* to move; momentum

and adaptive optimizers determine *how history changes that move*.